Laravel allows you to bind an interface to a concrete class, useful for creating flexible, testable code.

First, create an interface in `app/Contracts`:

```
namespace App\Contracts;

interface LoggerInterface {
    public function log(string $message);
}
```

Then, create a class that implements the interface:

```
namespace App\Services;

use App\Contracts\LoggerInterface;

class FileLogger implements LoggerInterface {
    public function log(string $message) {
        // write to file
    }
}
```

Next, bind the interface to the implementation in a service provider, typically `AppServiceProvider`:

```
public function register()
{
    $this->app->bind(LoggerInterface::class, FileLogger::class);
}
```

Now, Laravel can resolve `LoggerInterface` automatically with `FileLogger`.

5.6 Singleton Bindings

You can bind a class as a singleton so that only one instance is used throughout the app.

```
$this->app->singleton(AnalyticsService::class, function ($app) {
    return new AnalyticsService(config('analytics.api_key'));
});
```

5.7 Contextual Binding

When you need different implementations of an interface in different contexts.

```
use App\Contracts\LoggerInterface;
use App\Services\DatabaseLogger;
```